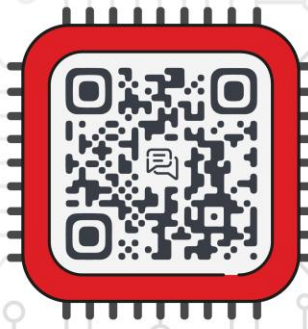




نظري

8

9

1080 sp

كلية الهندسة المعلوماتية

السنة الثالثة

Cache Memory

د. سيرا أستور



RB Informatics; 15/12/2025

بيان الحواسيب 2

مراجعة لأفكار المحاضرة السابقة :

لماذا نحتاج ال Cache Memory؟

المعالج (CPU) سريع جداً و الذاكرة الرئيسية (Main Memory / RAM) بطيئة نسبياً لذلك نضع ذاكرة صغيرة جداً وسريعة بين المعالج والذاكرة الرئيسية (Cache Memory) وهي تعتمد على 3 نقاط أساسية أهمها :

1. مبدأ المحلية (Locality Principle)

البرامج لا تستخدم كل الذاكرة دفعة واحدة، بل جزء صغير منها ولها نوعان :

1. Temporal Locality (المحلية الزمنية):

إذا استخدمت بيانات الآن فغالباً ستستخدمها مرة ثانية لذلك نحتفظ بالبيانات التي استخدمت مؤخراً في الكاش

مثال:

```
for(i = 0; i < 100; i++)
    sum += a;
```

متغير داخل حلقة for

تعليمية تنفذ بشكل متكرر

2. Spatial Locality (المحلية المكانية)

إذا استخدمت عنوان ذاكرة فغالباً ستستخدم العناوين القريبة منه

لذلك نحمل مجموعة كلمات متجاورة (Block) بدل كلمة واحدة

مثال:

مصفوفة array عندما تقرأ $A[5]$ غالباً بعدها $A[6]$ و $A[7]$

ولذلك الكاش لا يخزن كلمة وحدة فقط بل Block (عدة كلمات متجاورة).

Block:

أصغر كمية بيانات تنقل بين الكاش والذاكرة (مجموعة كلمات متجاورة) لماذا لا نستخدم Word واحدة فقط؟

في الكاش نعتمد على مبدأ:

Spatial Locality أي إذا استخدمت:

$Mem[101]$, $Mem[102]$, $Mem[103]$ غالباً ستستخدم: $Mem[100]$

فبدلاً من جلب كل كلمة لوحدها نجلبهم كلهم مرة وحدة أي نستفيد من هذا المبدأ ونوفر الوقت ونقل عدد الـ

Misses

مثال للفهم:

بفرض لدينا محل تجاري في الحريقة ونريد جلب بضائع إلى محل آخر فهنا نجلب كميات كبيرة ولا نقوم بأخذ بضاعة واحدة وكذلك العمل في ال ram

Miss and Hit:

المعالج يتعامل فقط مع الكاش لذلك عندما يطلب بيانات لا يذهب للذاكرة الرئيسية إلا إذا اضطر التسلسل:

Processor → Cache → Main Memory

لذلك حالات الوصول الى Cache هي Miss و Hit :

- Hit(1): البيانات موجودة في الكاش و القراءة أو الكتابة تتم مباشرة
- Miss(0): البيانات غير موجودة في الكاش فسيتم جلب Block كامل من الذاكرة الرئيسية ثم يكمل المعالج طلبه

ماذا يحتوي كل سطر في الكاش؟

كل Cache Line (سطر في الكاش) يتكوّن عادة من:

Valid Bit 1.

هذا البت يخبرنا: هل البيانات الموجودة في هذا السطر صالحة أم لا؟

إذا كان:

- Valid = 0 إذا: البيانات غير صالحة (الكاش فاضي أو قديم)
- Valid = 1 إذا: البيانات صحيحة ويمكن استخدامها

Tag 2.

هو جزء من عنوان الذاكرة

يُستخدم لمعرفة: هل البيانات المطلوبة موجودة فعلاً في هذا السطر أم لا؟

المعالج يقارن ال Tag القادم من العنوان مع ال Tag المخزن في الكاش

Cache Performance:

- ماذا يحدث عند طلب المعالج لبيانات؟

1. البحث في الكاش:

المعالج ينظر للكاش أولاً و يتحقق من:

1. Valid Bit و تطابق ال Tag

2. حالتان ممكنتان:

Cache Hit

البيانات موجودة في الكاش

الزمن المستغرق (T_1 = صغير جداً)

Cache Miss:

البيانات غير موجودة و يجب جلبها من الذاكرة الرئيسية كما ذكرنا سابقا ويصبح الزمن $T_1 + T_m$ (كبير) وبناء على ذلك تكون المعادلة :

$$ACT = T_1 + T_m \times (1 - h)$$

حيث:

- T_1 : زمن الوصول للكاش (صغير)
- T_m : زمن الوصول للذاكرة الرئيسية (كبير)
- H : Hit Rate (نسبة النجاح)
- Miss Rate = 1 - Hit Rate

ونستنتج انه كل ما زاد الـ Hit Rate، ACT (Average Access Time) سيصغر و الأداء يتحسن.

أنواع Cache Mapping :

1. Direct Mapped
2. Fully Associative
3. Set Associative

وفي هذه المحاضرة سنتعلم أول نوع Direct Mapped:**Cache / Main Memory Structure:**

كل Block في الذاكرة الرئيسية له مكان واحد فقط في الكاش وهذا يعني انه :

- إذا بلوك رقم 5 يجب الذهاب الى سطر معين
- إذا سطر الكاش مشغول فيتم استبداله مباشرة

تقسيم العنوان (Address Breakdown)

عنوان الذاكرة يُقسّم إلى ثلاثة أجزاء:

1. الإزاحة داخل الكتلة (Offset): يحدد البايت المطلوب داخل Block ويكون عدد البتات =

$$\log_2 (\text{حجم الكتلة})$$

2. (Index): يحدد خط الكاش المحدد

$$\log_2 (\text{الكاش خطوط عدد}) = \text{البتات عدد}$$

3. (Tag): وهي البتات المتبقية من العنوان و يستخدم للمقارنة مع Tag المخزن في الكاش

ملاحظة :

$$Index = Block Number \bmod Cache Lines$$

يعني:

- نأخذ رقم البلوك في الذاكرة
- نقسمه على عدد خطوط الكاش
- الباقي هو رقم سطر الكاش

:Example Direct Mapping ↩

لدينا :

$$Cache Size = 16 KB, Block Size = 4 Words, Word = 4 Bytes$$

$$Address Size = 32 bits$$

والمطوب حساب :

Offset bits/ Index bits/ Tag bits

- أولا : نحسب حجم البلوك بالبايت:

$$Block = 4 Words$$

$$Word = 4 Bytes$$

$$Block Size = 4 \times 4 = 16 Bytes$$

- ثانيا: حساب عدد Cache Lines :

$$Cache Size = 16KB = 16384 Bytes$$

- حيث:

$$1k = 1024, Block Size = 16 Bytes$$

$$Cache Lines = 16384 / 16 = 1024 = 1kb$$

أي الكاش فيه 1024 سطر

Tag	Index	Offset	Addresses
00000000000000000000	0000000001	0100	0X00000014
00000000000000000000	0000000001	1100	0X0000001C
00000000000000000000	0000000011	0100	0X00000034
00000000000000000010	0000000001	0100	0X00008014

- ثالثا:

:Offset Bits

$$Block Size = 16 Bytes$$

$$Offset = \log_2(16) = 4 bits$$

Index Bits

$$Cache Lines = 1024 Line$$

$$Index = \log_2(1024) = 10 bits$$

:Tag Bits

العنوان = 32 bits

$$Tag = 32 - (Index + Offset)$$

$$Tag = 32 - (10 + 4) = 18 \text{ bits}$$

إذن:

| Tag (18 bits) | Index (10 bits) | Offset (4 bits) |

■ سنقرأ العنوان الأول :

Address = 0x00000014

كل الـ Valid bits = 0 لان الكاش فاضية (فى بداية التشغيل)

تحويل العنوان إلى Binary

$0x14 = 0001\ 0100$ (*binary*)

- العنوان الكامل (32 bits) :

00000000000000000000 0000000001 0100

$$Tag = 00000000000000000000$$

Index = 0000000001

Offset = 0100

		Tag				Index	Offset
		00000000000000000000				0000000001	0100
Valid							
Index	Tag	0x0-3	0x4-7	0x8-b	0xc-f		
0	0						
1	0						

$$\text{Index} = 0000000001 = 1$$

استخدمنا:

المعالج يذهب مباشرة إلى Cache Line رقم 1، لم نقرأ البيانات فقط حددنا السطر

ثم يتم فحص محتوى السطر:

Cache Line = 1 ■

Invalid Data $\leftarrow$ Valid Bit = 0 ■

النتيجة؟ MISS (Cold Miss) لأن:

السطر لم يُستخدم من قبل لذلك البيانات غير صالحة ولا يمكن القراءة من الكاش

Valid Tag Index Offset
00000000000000000000 0000000001 0100

Index Tag 0x0-3 0x4-7 0x8-b 0xc-f

0	0				
<u>1</u>	0				

$$Index = 0000000001 = 1$$

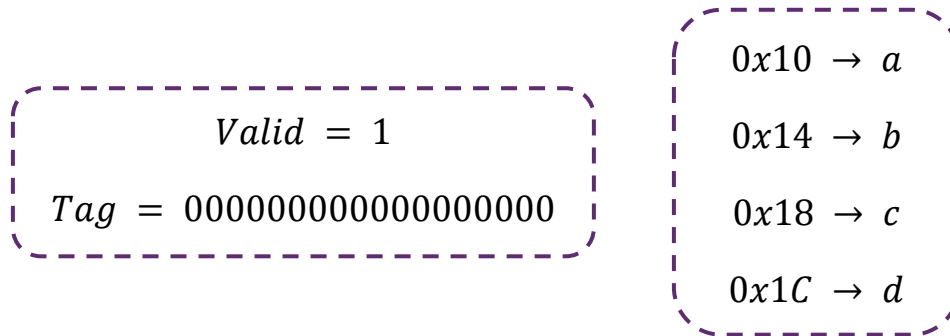
المعالج يذهب مباشرة إلى: Cache Line رقم 1

ماذا يحدث عند MISS؟

يتم جلب البلوك كامل من الذاكرة الرئيسية

البلوك يحتوي 4 كلمات:

تحميل البلوك في Cache Line 1



		Tag		Index		Offset
		00000000000000000000		0000000001		0100
Index	Valid	Tag	0x0-3	0x4-7	0x8-b	0xc-f
0	0					
1	1	0	a	b	c	d
2	0					

Valid = 1 و Tag تم تخزينه و البيانات موزعة حسب الـ Offset:

Offset Range	Data
0x0-3	A
0x4-7	B
0x8-b	C
0xc-f	D

الدائرة الحمراء على d: Offset = 0100 و يشير للكلمة المطلوبة داخل البلوك

■ قراءة العنوان الثاني:

$$Address = 0x0000001C$$

Binary: 00000000000000000000 0000000001 1100

Index = 1 / Offset = 1100 / Tag = السابق نفس

■ Cache Line نفس → Index = 1

▪ Tag / Valid = 1 مطابق

▪ القراءة مباشرة من الكاش / لا نذهب للذاكرة

Valid		0000000000000000 0000000001 1100			
Index		Tag	0x0-3	0x4-7	0x8-b
0	0				
1	1	0	a	b	c
2	0				
3	0				

▪ قراءة العنوان الثالث:

Address = 0x00000034

Index تغيير من 3 → 1

وهذا يعني:

Cache Line جديد ولكن نلاحظ ان Valid = 0 أي الحالة هي MISS

ماذا يحدث؟ يتم تحميل البلوك من الذاكرة :

- Cache Line 3 وضعه في
- Valid = 1
- Tag = 0
- عنوان الذاكرة يزداد كل مرة 4

x00000030
x00000034
x00000038
x0000003C

Valid		0000000000000000 0000000011 0100			
Index		Tag	0x3-0	0x7-4	0xb-8
0	0				
1	1	0	a	b	c
2	0				
3	1	0	e	f	g

حيث f هي المطلوبة حسب ال Offset

قراءة العنوان الرابع

Address = 0x00008014

تقسيم العنوان:

Tag = السابق عن مختلف

Index = 0000000001 = 1

Offset = 0100

أي نستنتج ان العنوان الجديد له نفس ال Index لكن Tag مختلف.

ماذا يفعل المعالج؟

تحديد Cache Line

نفحص حالة السطر:

Index = 1

والمعالج يذهب مباشرة إلى Cache Line رقم 1

Valid = 1 (السطر يحتوي بيانات)

العنوان الجديد Tag ≠ Tag المخزن

النتيجة : MISS (Conflict Miss)

■ لماذا Conflict Miss ؟ لأن Direct Mapped Cache :

فهذا يعني ان أكثر من Block في الذاكرة يمكن أن يملك نفس Index ولكن سطر واحد فقط متاح لهم في الكاش
ماذا يحدث عند الـ Conflict Miss ؟

يتم إزالة الب্লوك القديم من Cache Line 1 ويتم تحميل الب্লوك الجديد من الذاكرة الرئيسية:

0x00008010

0x00008014

0x00008018

0x0000801C

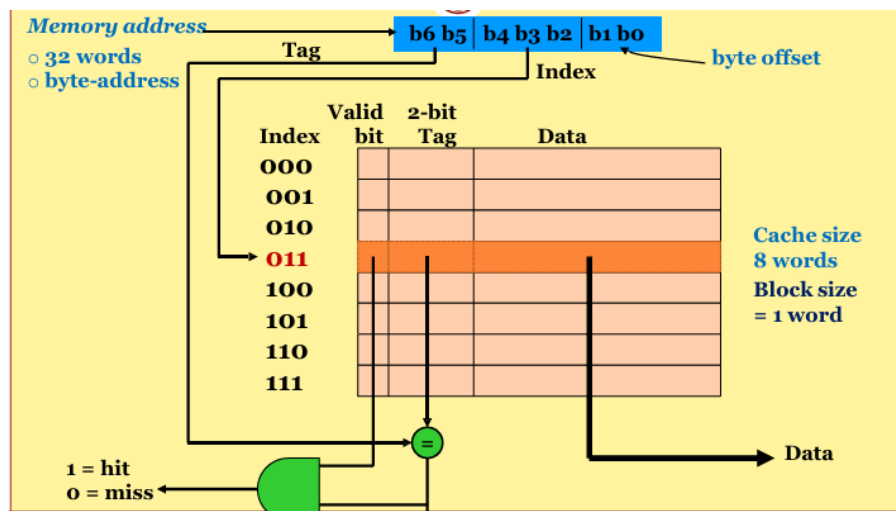
تحديث السطر من خلال : Tag = Tag الجديد و Valid = 1 فالبيانات أصبحت جاهزة في الكاش

$offset = 0100$

يشير إلى الكلمة المطلوبة داخل الب্লوك و القراءة تتم مباشرة من الكاش بعد التحميل

نفس Index + Tag مختلف

Valid		Tag		Index		Offset	
		000000000a0000000010		0000000001		0100	
Index		tag					
		0x0-3		0x7-4		0x8-b	
0	0						
1	1	2	i	j	k	l	
2	0						
3	1	0	e	f	g	h	
4	0						



ملاحظات :

- حجم الكاش محدود، لذلك عند جلب بيانات جديدة قد نضطر إلى استبدال بيانات قديمة (Cache Replacement)
- عند استبدال البيانات، لا يتم دائماً إرجاع البيانات القديمة إلى الـ RAM ، وإنما يعتمد ذلك على ما إذا كانت البيانات قد تم تعديلها أم لا لذلك لكاش يستخدم : Dirty Bit
- Dirty = 0 البيانات لم يتم تعديلها، لا حاجة لكتابتها في RAM.
- Dirty = 1 البيانات تم تعديلها، يجب كتابتها إلى RAM قبل الاستبدال
- الكاش يدعم عمليتي القراءة والكتابة (Read & Write) بهدف تقليل عدد مرات الوصول إلى الـ RAM وتحسين الأداء.
- تحسين الأداء باستخدام الكاش ليس دائماً ملحوظاً، خاصة إذا كان الوصول للذاكرة عشوائياً أو حجم البيانات أكبر من حجم الكاش، مما يؤدي إلى زيادة الـ Miss Rate.
- عند إقلاع الجهاز، تكون الذاكرة الرئيسية (RAM) غير مهيأة وتحتوي على قيم غير معروفة (قد تبدو عشوائية في لغة C++)
- عند طباعة قيم متغير لم تسند له قيمة تكون قيمته عشوائية)
- في لغة Java المتغيرات يتم تهيئتها تلقائياً (int = 0)
- Direct Mapping Pros & Cons
- Direct Mapping Pros:
- بسيط وسهل التصميم / رخيص في التنفيذ/ سرعة في الوصول
- Direct Mapping Cons:
- موقع ثابت لكل كتلة
- إذا كان البرنامج يصل لكتلتين مختلفتين تشاركان نفس الخط:
- سيحدث تضارب (Conflict)
- إخفاقات متكررة في الكاش
- انخفاض الأداء